第二章 线性表 · C语言知识准备

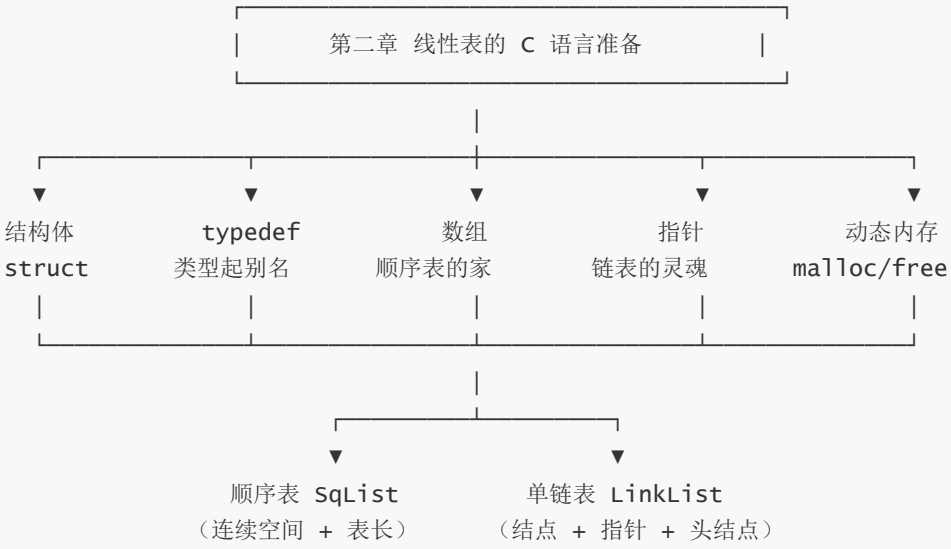
> 配套：《第二章 线性表的结构分析与应用-理论部分》《第二章 线性表的结构分析与应用-实践部分》
> 定位：在动手实现顺序表、单链表之前，先把要用到的 C 语言语法一次补齐。看完这章，你应该做到——**理论部分的代码看得懂，实践部分的代码能自己敲出来。**

0. 先看清"要补哪些 C 知识"

线性表本身是**逻辑结构**，要用 C 语言落地，必须依赖下面这些语法。先看这张对照表，做到心中有数：

线性表里的概念	需要的 C 语法	实践部分出现在哪里
数据元素（学号 + 成绩）	结构体 `struct`、`typedef`	`ElemType`
顺序表（数组 + 表长）	一维数组、结构体嵌套	`SqList { data[MAXSIZE]; length; }`
链表结点（数据 + 指针）	自引用结构体、结构体指针	`LNode { ...; struct LNode *next; }`
结点分散存放、随时增删	动态内存 `malloc` / `free`	`CreateListTail`、`ListInsert`
顺着链子往后走	指针移动 `p = p->next`	`GetElem`、`PrintList`
初始化、插入要"改表"	函数传址（指针参数）	`InitList(&L)`
被删元素要"带回来"	指针作输出参数	`ListDelete(&L, 3, &e)`
操作成功还是失败	用返回值 1 / 0 当信号灯	所有 `ListXXX` 函数
姓名这类文字	字符数组 + `strcpy` / `strcmp`	`char name[20]`

知识地图：



1. 结构体 `struct`：把"数据项"打包成"数据元素"

1.1 为什么需要它

理论部分说过：**数据元素**是基本单位，一个数据元素由若干**数据项**组成。比如"一名学生的成绩信息"包含学号和成绩两个数据项——它们类型不同，用数组装不下，这时就要用结构体把它们**打包成一个整体**。

1.2 定义与成员访问

```
// 定义结构体类型：一名学生的成绩信息
struct Student {
    long id;        // 成员（数据项）1：学号
    int  score;     // 成员（数据项）2：成绩
};

int main() {
    struct Student s1;      // 定义结构体变量
    s1.id = 1001;           // 用 "变量名.成员名" 访问
    s1.score = 92;
    printf("%ld %d\n", s1.id, s1.score);
    return 0;
}
```

> **总之：** 结构体就是一张"表格的表头"——规定了这张表每行有哪些列；结构体变量就是其中的**一行数据**。

1.3 结构体可以整体赋值（实践部分大量用到）

这是结构体最好用的特性：**同类型的结构体变量之间可以直接用 `=` 整体拷贝**，不必一个成员一个成员搬。

```
ElemType a, b;
a.id = 1001;  a.score = 92;
b = a;                  // 一次性把 id、score 全部复制过去
```

实践部分的两处代码正是靠这一点：

```
*e = L->data[i - 1];    // 顺序表删除：把整个元素备份出来
*e = *q;                // 单链表删除：把整个结点的数据域备份出来
```

1.4 结构体数组

结构体变量也能组成数组——**一个数组元素就是一个数据元素**，这正是顺序表的雏形：

```
ElemType data[100];     // 100 个学生成绩，连续存放
data[0].id = 1001;      // 先按下标取到结构体，再用 . 取成员
data[0].score = 92;
```

1.5 结构体嵌套：拼出完整的顺序表

```
#define MAXSIZE 100

typedef struct {
    long id;
    int  score;
} ElemType;

typedef struct {
    ElemType data[MAXSIZE];    // 数组：存放元素的"仓库"
    int      length;           // 计数器：当前有几个元素
} SqList;
```

> **关键理解：** `SqList` 不是元素本身，而是"**仓库 + 计数器**"。仓库是连续的（顺序存储的本质），计数器告诉我们在仓库里**有效数据到哪儿为止**——`data[length]` 及以后的格子是"空位"，是垃圾值，不能读。

---

2. `typedef`：给类型起个"外号"

2.1 基本用法

```
typedef 原类型名 新类型名;
```

```
typedef int ElemType;       // 以后 ElemType 就等于 int
ElemType x = 10;            // 等价于 int x = 10;
```

好处：哪天要把元素从 `int` 换成 `long`，只需改这一行，全程序不用动——这就是"数据抽象"的雏形。

2.2 结构体 + typedef 的固定套路

实践部分这两段写法必须看明白：

```
typedef struct {
    long id;
    int  score;
} ElemType;                 // ElemType = 这个结构体类型

typedef struct LNode {
    long id;
    char name[20];
    int  pms;
    struct LNode *next;
} LNode, *LinkList;         // 一次起了两个外号！
```

最后一行 `} LNode, *LinkList;` 同时定义了**两个**名字，含义完全不同：

写法	含义	用它声明变量时
`LNode`	结构体**类型**本身	`LNode e;` → e 是一个结点
`*LinkList`	指向该结构体的**指针类型**	`LinkList L;` → 等价于 `LNode *L;`，L 是一个指针

> **易错点：** `LinkList L` 里的 `L` 是**指针**，不是结点。所以函数参数写 `LinkList L` 时，函数内部改 `L->next` 能影响外面，但改 `L` 本身（让它指向别处）影响不了外面——这与 5.3 节的"传值/传址"是同一个道理。

---

3. 数组：顺序表的"连续空间"

3.1 下标从 0 开始 —— 位序 i ↔ 下标 i-1（第二章最易错点）

```
位序:     1     2     3     4     5        ← 人说话的习惯："第 i 个元素"
下标:     0     1     2     3     4        ← C 语言的习惯："data[i-1]"
        ┌─────┬─────┬─────┬─────┬─────┐
        | a1  | a2  | a3  | a4  | a5  |
        └─────┴─────┴─────┴─────┴─────┘
          ↑ 第 1 个元素存在 data[0]
```

牢记换算公式：第 i 个元素 → `L.data[i - 1]`

实践部分所有函数参数里的 `i` 都是**位序**（从 1 开始），代码里访问数组时全部减了 1：

```
L->data[i - 1] = e;              // 插入：第 i 个位置 → 下标 i-1
*e = L->data[i - 1];             // 删除：第 i 个位置 → 下标 i-1
return i + 1;                    // 查找：下标 i（从 0 起）→ 位序 i+1
```

3.2 遍历数组的标准写法

```
for (i = 0; i < L.length; i++) {   // 注意是 i < length，不是 i <= length
    printf("%ld\n", L.data[i].id);
}
```

3.3 C 语言不检查数组越界！

`data[100]` 却写 `data[100]`，编译器**不会报错**，程序可能当场崩溃，也可能悄悄改坏别的变量。所以顺序表的每个函数开头都要"判位置合法"：

```
if (i < 1 || i > L->length + 1) return 0;   // 插入：1 ≤ i ≤ length+1
if (i < 1 || i > L->length)     return 0;   // 删除：1 ≤ i ≤ length
if (L->length >= MAXSIZE)       return 0;   // 判表满
```

---

4. 字符数组与字符串

4.1 字符串的本质

C 语言没有 `string` 类型，字符串就是**以 `'\0'` 结尾的字符数组**：

```
char name[20];      // 最多装 19 个有效字符，最后一个位置留给 '\0'
```

4.2 赋值、比较必须用函数（对比表）

想做的事	错误写法	正确写法	原因
复制字符串	`s->name = e.name;`	`strcpy(s->name, e.name);`	数组名是常量地址，不能被赋值
比较字符串	`if (name == "张三")`	`if (strcmp(name, "张三") == 0)`	`==` 比的是地址，不是内容
输入字符串	`scanf("%s", &s->name);`	`scanf("%s", s->name);`	`name` 本身就是地址，**不用加 `&`**

```
#include <string.h>                 // strcpy / strcmp 都在 string.h 里
strcpy(e.name, "李四");
if (strcmp(e.name, "李四") == 0) {  // 相等时 strcmp 返回 0（记住这个反直觉的规定）
    printf("找到了\n");
}
```

5. 指针

5.1 一句话理解指针

指针是内存地址，

> **指针就是指内存地址。** 变量住在内存里，每间"房间"都有门牌号，指针保存的就是这个门牌号。

> **指针变量**是存放内存地址的变量

```
内存示意（每个格子是一个字节）：
地址:    0x1000   0x1004   0x1008
         ┌──────┬──────┬──────┐
内容:    |  10  |  20  | ...  |
         └──────┴──────┴──────┘
变量:       a        b

int a = 10;
int *p = &a;     // p 存储了 a 的地址 0x1000
```

指针变量的大小

```
printf("sizeof(char*) = %d\n", sizeof(char*));   // 4(32位)或8(64位)
printf("sizeof(int*)  = %d\n", sizeof(int*));
printf("sizeof(double*)= %d\n", sizeof(double*));
// 所有类型的指针变量大小相同（因为都存地址）
```

指针与整数的加减

```
#include <stdio.h>
int main()
{
    int arr[] = {10, 20, 30, 40, 50};
    int *p = arr;    // p指向arr[0]

    printf("*p = %d\n", *p);        // 10
    printf("*(p+1) = %d\n", *(p+1)); // 20
    printf("*(p+2) = %d\n", *(p+2)); // 30

    p++;     // p移动到下一个int，地址实际加了sizeof(int)=4
    printf("*p = %d\n", *p);        // 20

    return 0;
}
```

指针的比较

```
int *p = &arr[0];
int *q = &arr[3];

if (p < q)  // 比较地址大小
    printf("p指向的元素在q之前\n");
```

> **关键**：`p + n` 的实际地址偏移 = `p` 的地址 + `n × sizeof(指向的类型)`。

指针的差值

```
int *p = &arr[0];
int *q = &arr[4];

printf("相差元素个数: %d\n", q - p);  // 4
```

5.2 两个运算符

运算符	名字	作用	示例
`&`	取地址	拿到变量的地址	`&a` → 1000
`*`	解引用	按地址找到变量本身	`*p` → 10（p 指向的那个值）

```c
int a = 10;
int *p = &a;         // p 指向 a：p 里存 a 的地址
printf("%d\n", *p); // 10，通过地址读 a
*p = 20;             // 通过地址改 a
printf("%d\n", a);  // 20，a 真的被改了
```

5.3 为什么线性表的函数都要"传地址"

看这个对比实验，务必亲手跑一遍：

```c
#include <stdio.h>

void ByValue(int x)   { x = 100; }    // 传值：拿到的是副本
void ByAddress(int *x){ *x = 100; }   // 传址：拿到的是原变量的地址

int main() {
    int a = 10;
    ByValue(a);
    printf("传值调用后 a = %d\n", a);     // 10 —— 没变！
    ByAddress(&a);
    printf("传址调用后 a = %d\n", a);     // 100 —— 变了！
    return 0;
}
```

结论：

- **传值**（`SqList L`）：函数拿到的是**复印件**，改了白改，函数结束就丢；
- **传址**（`SqList *L`）：函数拿到的是**原件地址**，改动永久生效。

对应到实践部分：

```c
void InitList(SqList *L);                    // 要修改表 → 必须 &L
int  ListInsert(SqList *L, int i, ElemType e);   // 要修改表 → 必须 &L
int  LocateElem(SqList L, long id);          // 只读不改 → 可以传值
```

> 单链表的 `InitList(LinkList *L)` 用的是**二级指针**：因为初始化时要让主函数里的 `L` 本身指向新申请的头结点，只传 `LinkList L`（一级）改不了外面的 `L`。而插入、删除只改**结点里的内容**（`next`），传 `LinkList L` 就够了。

5.4 NULL 与野指针

```c
LNode *p = NULL;      // NULL = 空指针，表示"哪儿也不指"，值是 0
if (p == NULL) { ... }   // 用之前一定先判空
```

- 链表的**末尾标志**就是 `p->next == NULL`；
- 遍历循环条件必须写 `while (p != NULL)`，否则访问空指针会导致程序崩溃；
- `malloc` 后不判空、`free` 后继续使用 → 都是**野指针**，是 C 程序最常见的崩溃来源。

> **动手练：** 指针基础（`&`、`*`、传址、输出参数）请完成第 14 节练习 1 ~ 3（★ 必做）。

6. 结构体指针与 `->` 运算符

6.1 `->` 就是"指针版的小数点"

```
LNode *p;
(*p).id = 2001;      // 写法一：先解引用再取成员（括号不能省）
p->id   = 2001;      // 写法二：-> 运算符，等价且更常用
```

6.2 自引用结构体：链表结点的关键

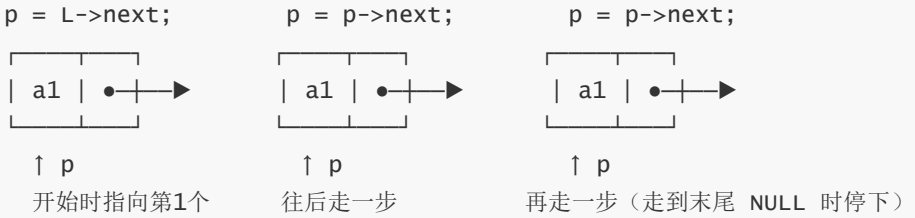
```
typedef struct LNode {
    long id;
    char name[20];
    int  pms;
    struct LNode *next;    // 指向"同类结点"的指针
} LNode, *LinkList;
```

为什么必须写 `struct LNode *next` 而不能写 `LNode next`？

- 指针大小固定（4 或 8 字节），编译器知道该分配多少空间；
- 如果写成 `LNode next`，就成了"结构体里套自己"，大小无限递归，编译直接报错。

> **一句话：** 结点用**指针**指向下一个结点，才把一个个独立的结点"串"成了链。

6.3 指针的移动：链表的"走路"方式




```
LNode *p = L->next;          // 从头结点的下一个（第一个真元素）开始
while (p != NULL) {          // 还没走到头
    printf("%ld\n", p->id);
    p = p->next;             // 顺着 next 往后挪一步
}
```

> **口诀：`p = p->next` 就是"走到下一个结点"，是链表所有操作的通用步伐。**

> **动手练：** 结构体指针、指针走数组、"先接后、再断前"请完成第 14 节练习 4、5（★★），学有余力再挑战练习 9。

7. 动态内存 `malloc` / `free`

7.1 为什么链表离不开它

数组长度要**编译期定死**（`data[100]`）；而链表希望**来一个结点申请一块空间**，表长随时变——这就要用到堆内存。

7.2 申请结点的"三件套"

```
#include <stdlib.h>

LNode *s;
s = (LNode *)malloc(sizeof(LNode));    // ① malloc 申请  ② sizeof 算大小  ③ 强转类型
if (s == NULL) {                       // ④ 一定判空：申请失败返回 NULL
    return 0;
}
```

组成部分	作用
`malloc(n)`	向系统申请 n 字节的连续空间，返回首地址（失败返回 `NULL`）
`sizeof(LNode)`	自动计算一个结点占多少字节，不用手数
`(LNode *)`	把 `void *` 强制转换成结点指针类型
`if (s == NULL)`	内存不足时的保护，避免后面访问空指针

7.3 用完必须还：`free`

```
free(q);        // 把这块空间还给系统
q = NULL;       // 好习惯：释放后立刻置空，防止野指针
```

- **忘记** `free` → 内存泄漏（程序跑久了越来越卡）；
- **重复** `free` 或 `free` **后继续使用** → 程序崩溃；
- 单链表删除结点时，顺序是：**先摘链（** `p->next = q->next` **），再备份数据，最后** `free(q)`。

7.4 结点的一生

```
malloc 申请  →  填数据  →  接入链表  →  摘链  →  free 释放
  (出生)       (上学)      (入队)      (出队)     (注销)
```

> **动手练：** `malloc` / `free` 请完成第 14 节练习 6（★★），二级指针可先跳过或只做选做的练习 7。

---

8. 函数：用返回值当"操作成功与否"的信号灯

8.1 返回 1 / 0

实践部分所有 `ListXXX` 函数都返回 `int`：

```
int ListInsert(SqList *L, int i, ElemType e) {
    if (位置不合法) return 0;      // 失败
    if (表已满)     return 0;      // 失败
    ...
    return 1;                      // 成功
}
```

调用方据此决定提示语：

```
if (ListInsert(&L, 2, e)) {
    printf("插入成功\n");
} else {
    printf("插入失败：位置不合法或表已满\n");
}
```

8.2 想"带回"多个结果 → 用指针当输出参数

C 函数只能返回一个值，删除操作既要知道**成功与否**，又要拿到**被删的元素**，怎么办？——返回值报状态，被删元素用指针参数"带回来"：

```
int ListDelete(SqList *L, int i, ElemType *e);   // e 是输出参数
...
ElemType e;
if (ListDelete(&L, 3, &e)) {          // 传 &e，函数里写 *e = ...
    printf("已删除：%ld %d\n", e.id, e.score);
}
```

8.3 参数到底加不加 `*`（速查表）

目的	参数写法	调用写法
只读取，不改	`SqList L` / `LinkList L`	`f(L)`
要改结构体内容	`SqList *L` / `ElemType *e`	`f(&L)` / `f(&e)`
要改指针本身（如初始化）	`LinkList *L`	`InitList(&L)`

9. 循环模板：三种必背写法

9.1 顺序表插入 —— 从后往前搬

```
for (j = L->length - 1; j >= i - 1; j--) {    // 从最后一个元素开始
    L->data[j + 1] = L->data[j];              // 逐个后移，腾出下标 i-1
}
L->data[i - 1] = e;
L->length++;
```

9.2 顺序表删除 —— 从前往后搬

```
*e = L->data[i - 1];                          // 先备份
for (j = i; j < L->length; j++) {             // 从被删位置的下一个开始
    L->data[j - 1] = L->data[j];              // 逐个前移，填上坑
}
L->length--;
```

9.3 单链表遍历 / 查找

```
LNode *p = L->next;          // 从第 1 个真元素开始
int j = 1;
while (p != NULL && j < i) {    // 两个条件：没走到尾 && 还没到目标位置
    p = p->next;
    j++;
}
return p;                    // p 为 NULL 表示位置 i 不存在
```

> **"差一错误"提醒**：`i < length` 还是 `i <= length`、从 `0` 还是从 `1` 开始，画一遍 3 个元素的示意图就能验算，不要凭感觉。

10. 输入输出格式速查

格式符	对应类型	示例
`%d`	`int`	`printf("%d", score);`
`%ld`	`long`	`printf("%ld", id);`
`%s`	字符串（`char []`）	`scanf("%s", name);` ← **不加** `&`
`%c`	单个字符	`scanf(" %c", &ch);`
`\t`	制表符（对齐用）	`printf("%ld\t%d\n", id, score);`

```
scanf("%ld,%d", &e.id, &e.score);    // 输入格式：1001,92（逗号必须原样输入）
scanf("%ld,%s,%d", &s->id, s->name, &s->pms);    // 输入：2001,张三,1
```

> **经验：** `scanf` 里的普通字符（如逗号）必须**原样输入**，否则读不到数据。更稳妥的做法是提示语里写清楚示例格式。

---

11. 高频错误对照表

错误写法	后果	正确写法
`InitList(L);`	函数改不动外面的表	`InitList(&L);`
`L.length = 0;`（L 是指针）	编译报错	`L->length = 0;`
`(*p).id` 写成 `*p.id`	编译报错（`.` 优先级高）	`(*p).id` 或 `p->id`
`s->name = e.name;`	编译报错（数组不可赋值）	`strcpy(s->name, e.name);`
`if (name == "张三")`	永远不相等	`if (strcmp(name, "张三") == 0)`
`scanf("%s", &name);`	类型不匹配	`scanf("%s", name);`
`for (i = 0; i <= L.length; i++)`	越界访问 1 个	`i < L.length`
`L->data[i]`（i 是位序）	位置整体错位	`L->data[i - 1]`
`malloc` 后不判空	内存不足时崩溃	`if (s == NULL) return 0;`
删除结点后不 `free`	内存泄漏	`free(q); q = NULL;`
先 `p->next = s;` 再 `s->next = p->next;`	后半条链丢失	**先接后，再断前**
遍历写 `while (p->next != NULL)`	漏掉最后一个结点	`while (p != NULL)`

---

12. 本章小结

知识点	关键记忆
结构体	打包数据项；同类型可整体赋值
`typedef`	`} LNode, *LinkList;` = 类型名 + 指针类型名
数组	位序 i ↔ 下标 i-1；C 不检查越界
指针	`&` 取地址、`*` 解引用；想改原件就传址
`->`	结构体指针取成员；`p = p->next` 往后走
`malloc` / `free`	申请必判空、释放必置空、用完必归还

知识点	关键记忆
函数设计	返回值报状态，指针参数带结果
字符串	复制 `strcpy`、比较 `strcmp`、`%s` 不加 `&`

---

13. 自测题（先做再对答案）

一、填空题

1. 定义结构体指针变量 `p` 后，访问其成员 `id` 应写作 `_____`。
2. 顺序表中"第 i 个元素"对应的数组下标是 `_____`。
3. 申请一个 `LNode` 结点的完整语句是 `s = (LNode *)_____(_____(LNode));`。
4. 想让函数修改主函数里的普通变量 `a`，调用时应写作 `f(_____)`。
5. 复制字符串用 `_____`，比较字符串用 `_____`，它们定义在头文件 `_____` 中。

二、程序阅读

写出下面程序的输出结果：

```
#include <stdio.h>
void swap1(int x, int y) { int t = x; x = y; y = t; }
void swap2(int *x, int *y) { int t = *x; *x = *y; *y = t; }
int main() {
    int a = 3, b = 5;
    swap1(a, b);
    printf("%d %d\n", a, b);
    swap2(&a, &b);
    printf("%d %d\n", a, b);
    return 0;
}
```

三、找错

下面这段"删除单链表第 i 个结点"的代码有 3 处错误，请指出并改正：

```
int ListDelete(LinkList L, int i) {
    LNode *p = GetElem(L, i - 1);
    LNode *q;
    if (p == NULL || p->next == NULL) return 0;
    q = p->next;
    p->next = q->next;
    return 1;
}
```

参考答案

一、填空题

1. `p->id`（或 `(*p).id`）
2. `i - 1`
3. `malloc`；`sizeof` → `s = (LNode *)malloc(sizeof(LNode));`
4. `&a`
5. `strcpy`；`strcmp`；`string.h`

二、程序阅读

```
3 5
5 3
```

（`swap1` 是传值，交换的只是副本；`swap2` 是传址，真正交换了 a、b）

三、找错

1. **没有备份数据**：被删结点的信息丢失，调用方拿不到。应加输出参数 `LNode *e`，并在摘链前 `*e = *q;`（也可以先摘链再备份，只要 `free` 之前备份即可）。
2. **没有释放内存**：缺少 `free(q);`，会造成内存泄漏，应补上 `free(q); q = NULL;`。
3. **参数设计不完整**：只有返回值 1/0，无法把被删元素"带回来"，正确原型为 `int ListDelete(LinkList L, int i, LNode *e);`。

---

14. 指针专项实践练习

> 目标：**先练熟指针，再写链表。**下面 9 个练习，每个只练**一个**指针动作，一步步走向顺序表和单链表。

怎么用这一节

1. **照着敲**：先把代码原样敲一遍并运行，看到结果再说，别急着改；
2. **改一改**：运行成功后再改数字、换数据，看输出怎么变（每道题都留了"改一改"）；
3. **答思考**：答不上来就回到正文对应小节再看一遍。

练习	难度	只练一件事	对应实践部分
1	★	`&` 取地址、`*` 解引用	看懂所有 `&L`、`*e`
2	★	传值 vs 传址	`InitList(&L)`、`ListInsert(&L, ...)`
3	★	指针作输出参数	`ListDelete(&L, 3, &e)`
4	★★	指针遍历数组	顺序表 `PrintList`
5	★★	`p = p->next` 走路	链表遍历 / `GetElem`
6	★★	`malloc` / `free`	`CreateListTail` 建表

练习	难度	只练一件事	对应实践部分
7	★★★	二级指针	`InitList(LinkList *L)`
8	★★	改错	全章易错点
9	★★★	先接后、再断前	`ListInsert` 核心一步

练习 1（★）：让指针指向 a——只做这一件事

任务： 照着敲、运行，看看 `*p` 和 `a` 是不是同一个东西。

```
#include <stdio.h>

int main() {
    int a = 10;
    int *p;              // p 是指针变量，专门用来装地址

    p = &a;              // &a 取出 a 的地址，装进 p —— 这就叫"p 指向 a"

    printf("a  = %d\n", a);
    printf("*p = %d\n", *p);    // *p：顺着地址找到的那个值

    *p = 20;             // 通过地址改 a
    printf("改之后 a = %d\n", a);
    return 0;
}
```

运行结果：

```
a  = 10
*p = 10
改之后 a = 20
```

要点（背下这三句就够了）：

- `&` 是**取地址**，`*` 是**按地址找人**；
- `p = &a` 之后：`p` 是"a 的地址"，`*p` 就是"a 本人"；
- 所以 `*p = 20` 和 `a = 20` 效果完全一样——改的是同一间房。

改一改（必做）： 把 `*p = 20;` 改成 `*p = *p + 5;`，先猜输出是多少，再运行验证。

选做（想看真实地址的同学）： 在 `return 0;` 前加一行：

```
printf("p 里的地址 = %p，a 的地址 = %p\n", (void *)p, (void *)&a);
```

两个值应该**完全一样**（数字每次运行可能不同，这很正常，别背数字）。

练习 2（★）：给成绩加 5 分——传值为什么"白改"

任务： 下面两个"加 5 分"的函数只有一个能真正生效。先猜输出，再运行验证。

```
#include <stdio.h>

void AddByValue(int score)   { score = score + 5; }   // 拿到复印件
void AddByAddress(int *score){ *score = *score + 5; } // 拿到原件地址

int main() {
    int s = 80;
    AddByValue(s);
    printf("传值后 score = %d\n", s);      // ?
    AddByAddress(&s);
    printf("传址后 score = %d\n", s);      // ?
    return 0;
}
```

预期结果：

```
传值后 score = 80
传址后 score = 85
```

要点（一句话记住）：

> 想让函数改掉外面的变量，就做两件事：**形参加 `*`，调用加 `&`**。缺一个都不行。

- 实践部分 `InitList(&L)`、`ListInsert(&L, i, e)` 就是这个道理；
- 只读不改的 `LocateElem(L, id)` 就不用加——**不改就不传址**。

改一改（必做）： 自己写一个 `void Swap(int *x, int *y)` 交换两名学生的成绩，在 `main` 里调用 `Swap(&s1, &s2)`，打印验证真的换了。

练习 3（★）：用指针把结果"带回来"（填空）

任务： 计算两个时刻的时间差。程序已写好，请把 4 个 `_____` 补上（答案就在题后，先别偷看）。

```
#include <stdio.h>

// h、m 是"输出参数"：函数结束后，主函数从这两个变量里拿结果
void TimeDiff(int start, int end, int *h, int *m) {
    int d = end - start;
    if (d < 0) d += 24 * 60;       // 结束早于起始，按跨天处理
    _____ = d / 60;                // ① 把"小时"写回主函数的变量
    _____ = d % 60;                // ② 把"分钟"写回主函数的变量
}

int main() {
    int h1, m1, h2, m2, h, m;
```

```
    printf("起始时间 HH:MM: ");
    scanf("%d:%d", &h1, &m1);
    printf("结束时间 HH:MM: ");
    scanf("%d:%d", &h2, &m2);

    TimeDiff(h1 * 60 + m1, h2 * 60 + m2, ______, ______);   // ③ 交出地址
    printf("相差 %d 小时 %d 分钟\n", h, m);
    return 0;
}
```

答案： ① `*h` ② `*m` ③ `&h, &m`

运行示例：

```
起始时间 HH:MM: 8:30
结束时间 HH:MM: 14:10
相差 5 小时 40 分钟
```

要点（为什么要这么麻烦）：

- 一个函数 `return` 只能带回**一个**值，而这里要"小时 + 分钟"两个 → 让指针帮忙带；
- 记住配对：**函数里写 `*h`（往地址里写），调用时写 `&h`（把地址交出去）**；
- 实践部分 `int ListDelete(SqList *L, int i, ElemType *e)` 一模一样：**返回值报告成功/失败，被删的学生靠 `&e` 带回来。**

改一改（必做）： 输入 `23:50` 和 `0:10`，看结果是不是 `0 小时 20 分钟`（跨天）。

练习 4（★★）：用指针走一遍数组（顺序表遍历的雏形）

4-1 必做：同一个遍历，两种写法

```
#include <stdio.h>

typedef struct {
    long id;
    int  score;
} ElemType;

int main() {
    ElemType L[3] = {{1001, 92}, {1002, 78}, {1003, 85}};
    int len = 3;
    int i;
    ElemType *p;

    printf("--- 写法一：用下标 ---\n");
    for (i = 0; i < len; i++) {
        printf("%ld\t%d\n", L[i].id, L[i].score);   // 下标用 . 取成员
    }

    printf("--- 写法二：用指针 ---\n");
    for (p = L; p < L + len; p++) {                 // p++ 走到下一个学生
```

```
        printf("%ld\t%d\n", p->id, p->score);        // 指针用 -> 取成员
    }
    return 0;
}
```

运行结果（两种写法输出完全一样）：

```
1001    92
1002    78
1003    85
```

要点（三条）：

1. 数组名 `L` 本身就是**首元素地址**，所以写 `p = L`，**不加** `&`；
2. `p++` 表示"走到下一个学生"（是整整跨过一个元素，不是加 1 字节）；
3. 指针取成员一律用 `->`，写 `p.id` 会**编译报错。**

改一改（必做）： 把第二段循环的 `p < L + len` 改成 `p <= L + len`，看看最后多打印出什么（这就是"越界读"，读到的是垃圾值）。

4-2 选做：看看一步跨多远

在 `return 0;` 前加：

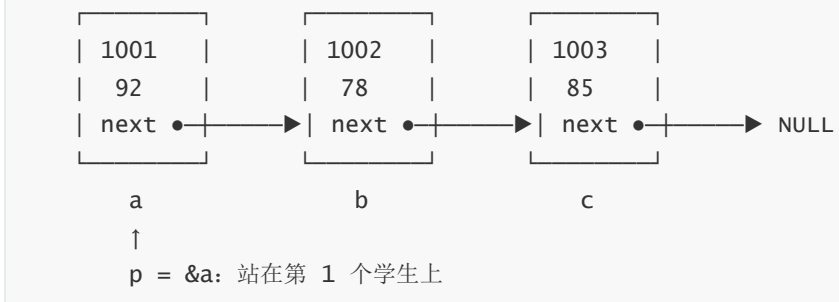
```
printf("L 的地址 = %p\nL+1 的地址 = %p\n一个元素占 %d 字节\n",
       (void *)L, (void *)(L + 1), (int)sizeof(ElemType));
```

两个地址之差正好等于 `sizeof(ElemType)`——这就是指针的**步长**，也是顺序表能"算地址直接定位"的原因。

练习 5（★★）：三个学生"手拉手"——先学会走路

先记住一句话： `p = p->next` 就是"走到下一个结点"。

先看图（三个普通变量，不用 `malloc`，手工拉手）：



第一步：先手工走三步（照着敲，体会 `p` 在移动）

```
#include <stdio.h>

typedef struct Node {
    long id;
```

```
    int  score;
    struct Node *next;      // 只能写指针，不能写 struct Node next
} Node;

int main() {
    Node a = {1001, 92, NULL}, b = {1002, 78, NULL}, c = {1003, 85, NULL};

    a.next = &b;            // a 拉住 b
    b.next = &c;            // b 拉住 c
                            // c.next 初始化时已是 NULL：队尾标志

    Node *p = &a;                                   // 站在第 1 个
    printf("第1个：%ld %d\n", p->id, p->score);
    p = p->next;                                    // 走到第 2 个
    printf("第2个：%ld %d\n", p->id, p->score);
    p = p->next;                                    // 走到第 3 个
    printf("第3个：%ld %d\n", p->id, p->score);
    return 0;
}
```

第二步：三行重复代码换成循环（这就是链表的遍历）

```
    Node *p = &a;
    while (p != NULL) {          // 还没走到队尾（NULL 就是队尾标志）
        printf("%ld\t%d\n", p->id, p->score);
        p = p->next;             // 走到下一个
    }
```

要点：

- 三个学生在内存里**挨不挨着无所谓**，是 `next` 把他们串起来的——这就是"链式存储"；
- 循环条件必须写 `while (p != NULL)`；写成 `while (p->next != NULL)` 会**漏掉最后一个学生**；
- 结点里只能写 `struct Node *next`（指针长度固定），写成 `struct Node next` 就成了"自己套自己"，编译直接报错。

改一改（必做）： 在循环里加一个计数器 `cnt`，走完后打印"共几名学生"。

思考： 走完之后 `p` 的值是什么？这时还能用 `p->id` 吗？（答：`p` 是 `NULL`，再访问就崩溃，所以循环里必须先判空。）

练习 6（★★）：结点的一生——`malloc` 申请，`free` 归还

任务： 申请一个新学生结点 → 填数据 → 打印 → 释放。照着敲一遍，记住这四步。

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    long id;
    int  score;
```

```
    struct Node *next;
} Node;

int main() {
    Node *s;

    s = (Node *)malloc(sizeof(Node));    // ① 申请一块"结点大小"的空间
    if (s == NULL) {                     // ② 一定判空（内存不足时返回 NULL）
        printf("内存申请失败！\n");
        return 1;
    }

    s->id = 1004;                        // ③ 填数据
    s->score = 88;
    s->next = NULL;                      //    新结点暂时不接任何人
    printf("新学生：%ld %d\n", s->id, s->score);

    free(s);                             // ④ 用完归还
    s = NULL;                            //    立刻置空，防止野指针
    return 0;
}
```

要点（四件套 + 一句口诀）：

- 申请四件套：`malloc` + `sizeof` + 强转 + 判空，**一个都不能少**；
- `malloc` 出来的结点在**堆**里，函数结束也不会自动消失，必须手动 `free`；
- 删除结点的顺序口诀：**先备份数据** → **再摘链** → **最后** `free`（顺序反了就再也拿不到数据）。

改一改（必做）： 把 `free(s);` 用 `//` 注释掉再运行——程序照样能跑，这就是**内存泄漏**（跑久了会越来越卡）。

练习 7（★★★ 选做）：为什么 `InitList(&L)` 要多加一个 `&`

先记住结论（实验只是验证它）：

- 只改**结点里的内容**（比如 `next`）→ 传一级指针 `Link L` 就够；
- 要改**指针变量自己**（让 `L` 指向新申请的头结点）→ 必须传二级 `Link *L`，调用写 `InitList(&L)`。

任务： 对比下面两个初始化函数，看主函数里的 `L` 到底有没有被改。

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node, *Link;

void BadInit(Link L) {                       // 一级：拿到的是指针的副本
    L = (Link)malloc(sizeof(Node));
    L->next = NULL;
}
```

```c
int InitList(Link *L) {                          // 二级：拿到的是"指针变量"的地址
    *L = (Link)malloc(sizeof(Node));
    if (*L == NULL) return 0;
    (*L)->next = NULL;
    return 1;
}

int main() {
    Link L = NULL;

    BadInit(L);
    printf("BadInit 之后 L = %p\n", (void *)L);    // 仍是 NULL —— 没改成功

    InitList(&L);
    printf("InitList 之后 L = %p\n", (void *)L);  // 有地址了 —— 成功
    printf("L->next = %p（NULL 表示空表）\n", (void *)L->next);

    free(L);
    L = NULL;
    return 0;
}
```

预期结果：

```
BadInit 之后 L = 0000000000000000
InitList 之后 L = 0000000000176C80
L->next = 0000000000000000（NULL 表示空表）
```

要点：

- 运行结果第一行是**全 0**（`NULL`），说明 `BadInit` 白干了——它改的是指针的**副本**；
- 实践部分 `InitList(LinkList *L)` 用的正是二级指针，而插入、删除只改结点内容，传一级就够了；
- **看不懂可以先跳过**：抄下来记住"初始化头结点要写 `InitList(&L)`"即可，等写完链表再回头看，会豁然开朗。

练习 8（★★）：改错（先自己找，再看答案）

任务： 下面这段"把新结点插到头结点之后"的代码有 4 处错误，请指出并改正。

给三个提示（都是本章正文讲过的）：

1. `malloc` 申请完，接下来**必须先做什么**？
2. 接链的两句话，**哪句该先写**？
3. 遍历循环条件到底写 `p != NULL` 还是 `p->next != NULL`？指针取成员用 `.` 还是 `->`？

> 这是一段**代码片段**，把它复制到练习 7 程序的类型定义后面就能编译运行（改之前先跑一次，看看它错在哪）。

```c
void AddNode(Link L, int x) {
```

```
    Node *s;
    s = (Node *)malloc(sizeof(Node));
    s->data = x;
    L->next = s;                    // 想把 s 接到头结点后面
    s->next = NULL;
}

void PrintList(Link L) {
    Node *p = L->next;
    while (p->next != NULL) {       // 遍历输出
        printf("%d ", s.data);
        p = p->next;
    }
}
```

参考答案：

#	错误	后果	改法
1	`malloc` 后没判空	内存不足时访问空指针崩溃	`if (s == NULL) return 0;`
2	先写 `L->next = s` 再写 `s->next = NULL`	原来的整条链被弄丢	**先接后，再断前**：`s->next = L->next;` `L->next = s;`
3	`s.data`	`s` 是指针，编译报错	`p->data`
4	`while (p->next != NULL)`	漏掉最后一个结点	`while (p != NULL)`

改正后的核心三行：

```
s->next = L->next;      // ① 先接后：新结点抓住后面的链
L->next = s;            // ② 再断前：头结点改指向新结点
```

练习 9（★★★ 挑战）：完成一次"链表插入"（先接后、再断前）

任务： 接着练习 5 的那条链 `1001 -> 1002 -> 1003`，把新学生 `x`（1004 / 90）插到**第 2 个位置**（1001 和 1002 之间）。要求**只改指针、不搬数据**，请把两处 `_____` 补上。

```
#include <stdio.h>

typedef struct Node {
    long id;
    int  score;
    struct Node *next;
} Node;

void Print(Node *head) {
    Node *p = head;
```

```
    while (p != NULL) {
        printf("%ld -> ", p->id);
        p = p->next;
    }
    printf("NULL\n");
}

int main() {
    Node a = {1001, 92, NULL}, b = {1002, 78, NULL}, c = {1003, 85, NULL};
    Node x = {1004, 90, NULL};

    a.next = &b;  b.next = &c;      // 先拉成一条链

    Print(&a);                      // 插入前

    ______ = ______;                // ① 先接后：x 抓住 a 原来后面的结点
    ______ = ______;                // ② 再断前：a 改指向 x

    Print(&a);                      // 插入后
    return 0;
}
```

答案： ① `x.next = a.next;`　② `a.next = &x;`

运行结果：

```
1001 -> 1002 -> 1003 -> NULL
1001 -> 1004 -> 1002 -> 1003 -> NULL
```

要点：

- 插入一个结点**只改了 2 个指针**，数据一个字节都没搬——这就是链表插入比顺序表快的原因（顺序表要整体后移）；
- 两句**绝对不能颠倒**：先写 `a.next = &x` 的话，就再也找不到 `b` 了，后半条链丢失；
- 把这里的 `x` 换成 `malloc` 申请的结点，再补上"找前驱"这一步，就是实践部分完整的 `ListInsert`。

过关自查（能答出来就可以去写实践部分了）

1. `p = p->next` 到底改变了谁？（答：改变的是 `p` 自己指向的位置，结点本身没动）
2. 遍历时为什么写 `while (p != NULL)` 而不是 `p->next != NULL`？（答：后者会漏掉最后一个结点）
3. 删除结点的正确顺序是什么？（答：备份数据 → `p->next = q->next` 摘链 → `free(q)`）
4. 为什么 `InitList(&L)` 要传 `&`，而 `ListInsert(L, ...)` 不用？（答：前者要改指针变量本身）